

Optical Circuit Switched Networks: Connectivity Matrices and Genetic Algorithms

Oneal Wang

October 2025

1 Shale

V is represented as a matrix of the sending node as rows and receiving node as columns

$$V = \mathcal{M}_{n \times n}$$

D is the sending data represented as a matrix with b bits and n nodes

$$D = \mathcal{M}_{b \times n}$$

T^i transformation from V to W for some timeslot i (cyclical) The full edge basis is trivial: $e_{i,j}$ is the standard unit vector with 1 at the coordinate for edge (i,j) and 0 elsewhere. Those $m = (nC2)$ vectors span R^m . The round-robin matching vectors are then simply sums of some of those edge unit vector s.

$$T^i = \mathcal{M}_{n \times n}$$

$$T^i \rightarrow \mathcal{L}(V, W)$$

W is the recieving node for each sending node for a specific time slot called an adjacency matrix

$$W = \mathcal{M}_{n \times n} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

S is the recieved data represented as a matrix with b bits and n nodes

$$S = \mathcal{M}_{b \times n}$$

1. Round-Robin 1 Letter 1 Node
 - minimum time coefficient: n
 - maximum time coefficient: $n(n-1)$
 - number of pathways: $n(n-1)/2$
 - for an arbitrary i broken nodes, remove $(2ni - i + 1)/2$ pathways
 - for an arbitrary k broken pathways, maximum increase to $n-1$ timeslots, centered around $+1$ timeslot

$$\mathcal{M}_{b \times n} = DT^i(V) = DW = S, i \in \{1, \dots, n-1\}$$

Example 2.1 (6 nodes, 5 timeslots, h=1):

W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 1 \\ 3 & 4 & 5 & 6 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 6 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$W_{5 \times 5} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$DT^i(V_{6 \times 6}) = DW_{6 \times 6} = D \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} = S, i \in \{1, \dots, 5\}$$

minimum time coefficient: 6

maximum time coefficient: 30

number of pathways: 15

2. Round-Robin 1 Letter repeat for every X Letters of 1 Node

number of nodes: $L^x = n$

timeslot number: $(L-1)^x$

number of pathways: $L^x \times (L-1)^x / 2$

representation in base L for matrix algebra

for each node k, available timeslots are exactly one digit difference from k in base L

for an arbitrary i broken nodes, remove $i(L-1)^x$ pathways

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{0, \dots, (L-1)^x\}, j \in \mathbb{Z}, 0 \leq j \leq L^x - 1$$

always multiple receivers for each timeslot so order within each column does not matter

Example 2.2 (3 letters, $x = 2$):

number of nodes: $3^2 = 9$

timeslot number: $2^2 = 4$

number of pathways: $3^2 \times 2^2 / 2 = 18$

W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 2 & 0 & 1 & 2 \\ 2 & 2 & 1 & 4 & 3 & 3 & 3 & 4 & 5 \\ 3 & 4 & 5 & 5 & 5 & 4 & 7 & 6 & 6 \\ 6 & 7 & 8 & 6 & 7 & 8 & 8 & 8 & 7 \end{bmatrix}$$

$$W_{8 \times 8} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathcal{M}_{b \times n} = DW = S, i \in \{1, \dots, n-1\}$$

3. Round-Robin t Letter repeat (Hamming-distance) for every X Letters of 1 Node

number of nodes: $L^x = n$

timeslot number: $(L-1)^{x+t-1}$

number of pathways: $L^x(L^x-1)^{x+t-1}/2$

representation in base L for matrix algebra

for each node k, available timeslots are exactly t digit difference from k in base, also the hamming distance L

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{0, \dots, (L-1)^{x+t-1}\}, j \in \mathbb{Z}, 0 \leq j \leq (L-1)^{x+t-1}$$

always multiple receivers for each timeslot so order within each column does not matter

Example 2.3 (4 letters, $x = 3$, $t=2$):

number of nodes: $4^2 = 16$

timeslot number: $3^3 = 27$

number of pathways: $4^2 \times 3^3 / 2 = 216$

W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A_{27 \times 27} = \begin{bmatrix} 5 & 4 & \dots & 2 & 3 \\ 6 & & & & 7 \\ \dots & & \dots & & \dots \\ 56 & & & & 57 \\ 60 & 61 & \dots & 59 & 58 \end{bmatrix}$$

$$W_{27 \times 27} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathcal{M}_{b \times 27} = DW = S, i \in \{1, \dots, n-1\}$$

1.1 Numerical Results

Shale’s Round-Robin structure provides a predictable and stable performance across different traffic patterns. Using the waterfilling algorithm, we observe that Shale effectively utilizes available pathways, though it may not reach the peak efficiency of more dynamic architectures under extreme skew.

1.1.1 Advanced Congestion Control and VLB Routing

To handle real-world traffic demands, our implementation of Shale incorporates:

- **Interleaved Schedules:** Simulation of parallel matching schedules to support both low-latency ($h = 1$) and high-throughput ($h = 4$) traffic streams.
- **Valiant Load Balancing (VLB):** A two-phase routing scheme where cells are first sprayed to intermediate nodes (Spraying Phase) before being routed directly to the destination (Direct Phase). This spreads the load and reduces hotspots.
- **PIEO Queues:** Push-In Extract-Out queues allow switches to extract the highest-priority eligible cell, overcoming Head-Of-Line (HOL) blocking when the physical link for the top cell is not currently connected in the timeslot.
- **Credit-Based Flow Control:** Tokens are exchanged between switches to manage buffer space (credits), preventing bufferbloat and ensuring hop-by-hop reliability.

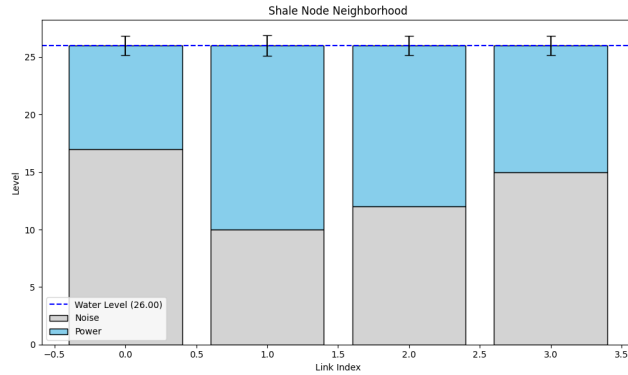


Figure 1: Shale Rigorous Waterfilling Performance

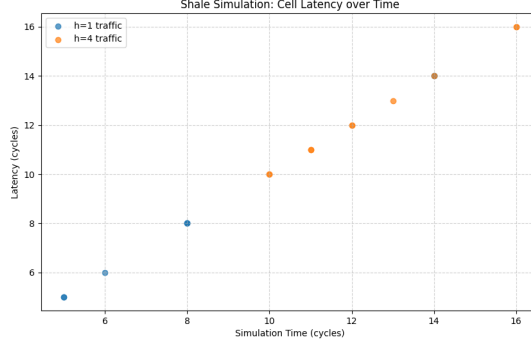


Figure 2: Shale VLB Simulation: Latency vs. Throughput under Interleaved Schedules

2 Opera

V is represented as a matrix of the racks with the sending node as rows and receiving node as columns

$$V = \mathcal{M}_{n \times n}$$

D is the sending data represented as a matrix with b bits and n nodes

$$D = \mathcal{M}_{b \times n}$$

1. Direct Path

timeslots = maximum number of hops for an arbitrary node

for an arbitrary i broken racks, maximum of $n-1$ hops

Example 3.1 (8 ToR Switches, 4 Rotor Circuit Switches):

A = represents a directed graph where each original row is the node and each column is the rack

$$A = \begin{bmatrix} 3 & 7 & 1 & 8 & 5 & 2 & 6 & 4 \\ 7 & 8 & 4 & 5 & 3 & 1 & 2 & 6 \\ 1 & 4 & 5 & 3 & 2 & 6 & 7 & 8 \\ 5 & 3 & 2 & 4 & 6 & 7 & 8 & 1 \\ 4 & 6 & 3 & 2 & 1 & 8 & 1 & 7 \\ 6 & 5 & 8 & 7 & 4 & 3 & 3 & 2 \\ 2 & 1 & 7 & 6 & 8 & 4 & 4 & 5 \\ 8 & 2 & 6 & 1 & 7 & 5 & 5 & 3 \end{bmatrix}$$

W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$W_i = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{i_{A_{i,k},i}}) = 0$$

2. (Uniform) Indirect Path

This uses BFS to find the optimal path from node a to node b.

Minimum of 1 hop and maximum of $\lceil \frac{n}{2} \rceil$ hops

For an arbitrary i broken racks, at least no hop up to +i hops change
number of timeslots = maximum number of hops

Example 3.2:

For the same A as example 3.1, suppose Rack 2 is broken.

- (a) The 1-hop path [0, 7] (via rack 2) is now impossible.
- (b) BFS checks other neighbors of 0. Let's say it finds node 6 (A[0][7] = 6, via rack 7).
- (c) BFS then checks neighbors of 6. (A[6][4] = 7). This is node 7, via rack 4.
- (d) Rack 4 is not broken, so this path is valid. Expected: [0, 6, 7], Rack: [0, 7, 4] (or another valid 2-hop path)

3. Weighted Indirect Path

This uses Dijkstra's Algorithm to find the optimal path from node a to node b (not necessarily the shortest).

Minimum of 1 hop and maximum of $\lceil \frac{n}{2} \rceil$ hops

timeslots = number of hops

For an arbitrary i broken racks, at least no hop up to +i hops change
number of timeslots = maximum number of hops

θ_k is the weight of the list where for each element, it is the weight of the
corresponding rack (constant if equal weight)

broken racks can be represented by ∞ weight

Example 3.3:

$$W_{8 \times 8} = \mathcal{M} \mid W_{A_{i,k},i} = \theta_k, \neg(W_{A_{i,k},i}) = 0$$

2.1 Numerical Results

Our analysis of the Opera architecture using the waterfilling algorithm confirms its efficiency in low-latency environments. However, the performance degrades as the number of hops increases, highlighting a significant multi-hop routing penalty.

As seen in Fig 3, Opera achieves near-peak capacity at low latencies, but the introduction of additional hops (Fig 2) introduces bottlenecks that the water-filling algorithm must navigate, leading to a more distributed power allocation.

3 Sirius

$\lambda_1, \dots, \lambda_i, \dots, \lambda_j$ = wavelength, corresponding timeslot

$P_1, \dots, P_k, \dots, P_l$ = ports

$G_1, \dots, G_p, \dots, G_q$ = gratings (AWGRs)

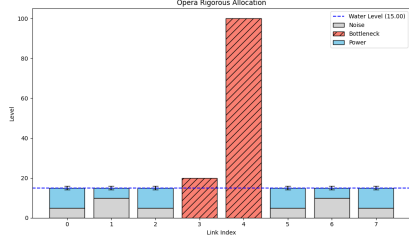


Figure 3: Opera Rigorous Waterfilling

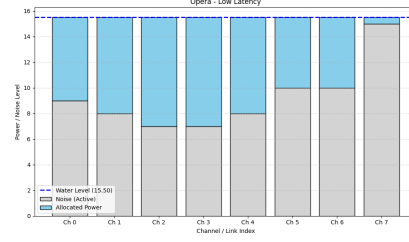


Figure 4: Opera Low Latency Performance

$N_1, \dots, N_s, \dots, N_r = \text{nodes}$

$r = j \times l$ (derivation is the pathways for an arbitrary node)

$jq = rl$ (derivation is the total number of pathways)

Then, it follows that $q = l^2$

Parent Function:

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{1, \dots, j\}$$

Source Matrix: $\mathcal{M}_{q \times l} \mid$

$$W_i =$$

Example 4.11 (2 wavelengths, 3 ports, 9 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 & 13 \\ 4 & 10 & 16 \\ 2 & 8 & 14 \\ 5 & 11 & 17 \\ 3 & 9 & 15 \\ 6 & 12 & 18 \end{bmatrix}, A^2 = \begin{bmatrix} 4 & 10 & 6 \\ 1 & 7 & 13 \\ 5 & 11 & 17 \\ 2 & 8 & 14 \\ 6 & 12 & 18 \\ 3 & 9 & 15 \end{bmatrix}$$

$$W_{8 \times 8}^1 = \mathcal{M} \mid W_{A_{i,k}^1, i} = 1, \neg(W_{A_{i,k}, i}) = 0, W_{8 \times 8}^2 = \mathcal{M} \mid W_{A_{i,k}^1, i} = 1, \neg(W_{A_{i,k}, i}) = 0$$

Example 4.12 (3 wavelengths, 2 ports, 4 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 \\ 3 & 9 \\ 5 & 11 \\ 2 & 8 \\ 4 & 10 \\ 6 & 12 \end{bmatrix}, A^2 = \begin{bmatrix} 3 & 9 \\ 5 & 11 \\ 1 & 7 \\ 4 & 10 \\ 6 & 12 \\ 2 & 8 \end{bmatrix}, A^3 = \begin{bmatrix} 5 & 11 \\ 1 & 7 \\ 3 & 9 \\ 6 & 12 \\ 2 & 8 \\ 4 & 10 \end{bmatrix}$$

3.1 Numerical Results

Sirius leverages wavelength-division multiplexing across time slots. Our simulation shows dynamic power allocation across these slots allows for high throughput, though scheduling complexity increases with the number of wavelengths. The waterfilling algorithm effectively balances the power across available links and wavelengths to maximize total Shannon capacity.

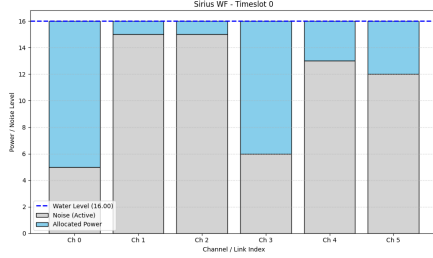


Figure 5: Sirius Waterfilling (TS 0)

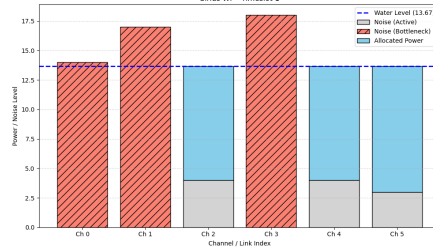


Figure 6: Sirius Waterfilling (TS 1)

Figures ?? and ?? illustrate the dynamic nature of power allocation in Sirius. Depending on the timeslot and the specific AWGR configuration, the capacity bottlenecks shift, requiring the waterfilling algorithm to adapt the power levels to maintain optical efficiency.

4 Genetic Algorithm

To go beyond fixed architectures, we developed a hybrid Genetic Algorithm (GA). The goal is to evolve a topology W that maximizes fitness:

$$Fitness = \alpha \cdot Capacity(W, T) + \beta \cdot \frac{1}{ASPL(W)} - \gamma \cdot Penalty(Degree)$$

where T is the traffic demand (e.g., skewed or hotspot traffic). We often initialize the population with a "Frozen Ring Backbone" to ensure basic connectivity (cycle graph) and let the GA evolve long-range shortcuts.

4.1 Numerical Results

The GA-evolved topologies were tested against stochastic waterfilling to ensure robustness. The results show that the GA successfully identifies long-range shortcuts that minimize the average shortest path length (ASPL) while maximizing Shannon capacity for the target traffic.

As shown in Figure ??, the GA architecture maintains high performance even under latency constraints, as it prioritizes direct links for high-traffic pairs evolved during the optimization phase.

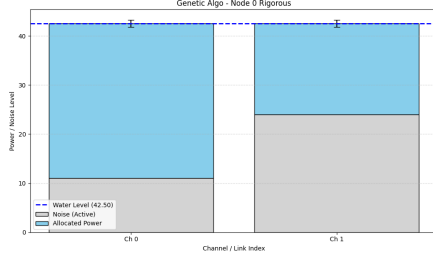


Figure 7: GA Rigorous WF (Node 0)

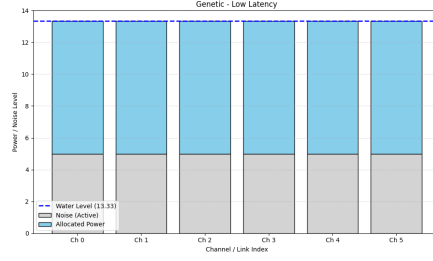


Figure 8: GA Low Latency

5 Performance Comparison

We benchmarked all architectures—Opera, Shale, Sirius, and the Genetic Algorithm—under a skewed traffic pattern. The results, summarized in Figure ??, demonstrate the clear advantage of topology evolution. While static architectures like Opera and Shale perform well under uniform assumptions, they struggle with highly non-uniform demands where specific node pairs dominate the traffic.

The Genetic Algorithm, by evolving its connectivity specifically for the skewed traffic matrix, effectively ”pre-allocates” capacity where it is most needed, resulting in significantly higher total Shannon capacity for the same aggregate power budget.

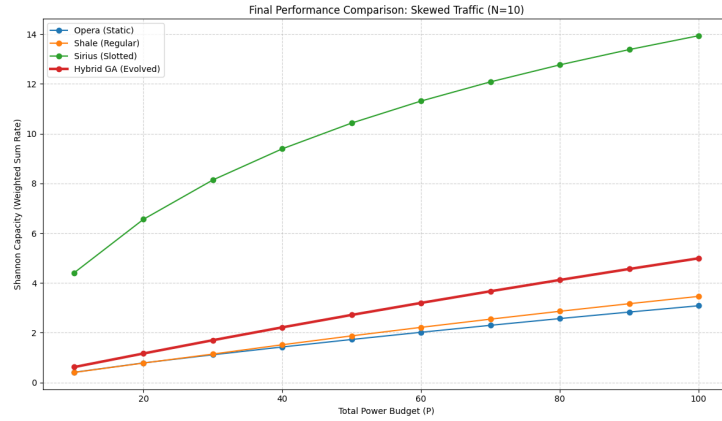


Figure 9: Architecture Comparison under Skewed Traffic using Shannon Capacity